

instant-ttc-fusion

Steps, Logic, Inputs, Outputs & Evaluation Criteria — Full Reference

Real-time LiDAR + Radar + Camera fusion for Time-to-Collision (TTC) estimation, evaluated on nuScenes-mini.
17 build-order steps · 139 automated tests · single held-out final evaluation

How to read this document

Each step below is documented the same way: its **purpose** in plain language, the **logic/algorithm** actually used (and why, where a real design choice was made), its **inputs** and **outputs**, and the **metrics or evaluation criteria** used to check it worked. Steps are listed in build order (not file-name order) — the order the project was actually built and proven in, which is why sensor adapters (steps 14–16) come near the end: they only make sense once the pipeline they feed already exists and is proven on synthetic data.

STEP 1 —
step00

Config, Contracts & Dataset Preflight

APPROVED

Purpose. Establish the single source of truth for every tunable constant and the shared data contracts every later stage passes between each other, and confirm the real dataset is actually present in the expected shape before any pipeline code depends on it.

Logic used

- Every constant lives in one **Param(value, status, reason, tunable_on)** record; status is one of **MEASURED / ASSUMED / PLACEHOLDER** — never a bare number with no stated justification.
- Shared dataclasses (**RawEvent**, **Detection**, **TrackSnapshot**, **TTCResult**, **ActionDecision**) defined once, reused by every later step — no step invents its own local data shape.
- Dataset preflight walks the real nuScenes-mini folder structure and confirms scenes, samples, and sweep counts on every active channel before anything else trusts it.

Inputs

- Raw nuScenes-mini dataset directory (samples/, sweeps/, v1.0-mini/*.json)

Outputs

- **src/ttcf/config.py** (the constant registry)
- **src/ttcf/types.py** (shared dataclasses)
- Preflight console report (scene/sample/sweep counts per channel)

Metrics / evaluation criteria

- Not a scored step — verified by unit tests (12/12) plus a real-data preflight check: 10 scenes, 404 samples, sweeps confirmed on all 12 channels, devkit version 1.2.0.

STEP 2 —
step02

Sensor → Ego → Global Coordinate Transforms

APPROVED

Purpose. Every sensor reports positions in its own local frame. Comparing anything across time or across sensors requires one common frame first — get this wrong and a parked car appears to move at the ego vehicle's own speed.

Logic used

- Built directly on the nuScenes devkit's own **transform_matrix** + **pyquaternion** — never a hand-rolled quaternion/rotation implementation.
- **FrameContext** bundles exactly one event's own **calibrated_sensor** + **ego_pose** records — no function anywhere can accidentally borrow another channel's pose.

Inputs

- A point (or batch of points) in one frame
- The event's own **calibrated_sensor** + **ego_pose** records

Outputs

- The same point(s) transformed into the target frame (sensor↔ego↔global)

Metrics / evaluation criteria

- Synthetic round-trip tests (transform then inverse recovers the original point exactly)
- Real-data ego-motion evidence check — found and fixed a real GT mislabelling bug (a nuScenes instance tagged vehicle.parked was actually moving ~5.4 m/s).

STEP 3 —

step03

Shared Kalman Filter & Ego-State Estimator

APPROVED

Purpose. One constant-velocity Kalman filter design, used identically to estimate both the ego vehicle's own velocity and every tracked object's velocity — not two separate implementations.

Logic used

- **ConstantVelocityKF**: predict/update with Joseph-form covariance update; internal clock stored as exact integer microseconds (a float-seconds version was found to drift).
- **EgoStateEstimator** derives ego velocity causally from the KF fed by real ego_pose history — never a raw finite difference between two GPS fixes.

Inputs

- A sequence of (t_us, position, R) measurements
- A query timestamp

Outputs

- **EgoState** / track state: (x, y, vx, vy, speed, heading) at the queried instant, with full covariance

Metrics / evaluation criteria

- Synthetic tests against known analytic constant-velocity cases
- Performance: an O(n) rebuild-from-scratch bug was found and fixed (43s after fix vs. a 3-minute-plus stall before).

STEP 4 —

step07A

TTC Math (Pure Formula)

APPROVED

Purpose. The actual time-to-collision arithmetic, kept as pure, side-effect-free math so it can be reused identically by the ground-truth table and the live tracked-object estimator.

Logic used

- **closing_speed** = ((ego_v - obj_v) · (obj - ego)) / distance; **TTC** = distance / closing_speed, with deadbands for near-zero object speed and near-zero closing speed (returns $TTC=\infty$ rather than a meaningless huge finite number).
- A genuine sign-convention error was found in this step's own written specification (contradicted its own worked example) — verified against test data and corrected in the source specification itself, not just the code.

Inputs

- Object reference position + velocity (global frame)
- Ego position + velocity (global frame)
- Optional object velocity covariance

Outputs

- **TTCMathResult**: distance_m, closing_speed_mps, ttc_s, reason code, closing_std_mps

Metrics / evaluation criteria

- 7 synthetic worked-example tests
- Frame-invariance check (same physical scenario, different coordinate origins, identical TTC)

Purpose. Define the ego vehicle's own driving corridor precisely, plus a deliberately wider "candidate" version so an object about to enter the corridor is never missed just before it matters.

Logic used

- Corridor is straight in the ego frame: $0 < x \leq \text{max_range}$, $|y| \leq \text{half_width}$.
- **candidate_gate** widens the half-width by a margin — the generous, position-only pre-tracker filter (DEC-3).
- GT/track membership uses the **nearest point on the object's own footprint** to the ego origin (DEC-1), not the footprint's centroid or a simple corner check.

Inputs

- A reference point (or a 4-corner footprint) in the ego frame

Outputs

- Boolean corridor/candidate membership
- The nearest point on a footprint, if applicable

Metrics / evaluation criteria

- Synthetic geometry tests (points inside/outside/exactly on every boundary)

Purpose. Build the answer key the whole project will be scored against — before any detector or tracker exists — using the exact same corridor/TTC/ego-state code the live pipeline will use later, so the comparison is apples-to-apples.

Logic used

- One row per (keyframe, in-path real GT object). GT velocity comes from the devkit's own **nusc.box_velocity()** — never a hand-rolled finite difference.
- Also proposes the fixed tune/eval scene split (DEC-8): 5 scenes each, with the one scene holding all 5 real AEB events deliberately assigned to eval.

Inputs

- nuScenes sample_annotation records, ego_pose history, the GT category scope (DEC-2)

Outputs

- **outputs/gt_events.csv** (315 rows)
- Per-scene summary table
- **configs/split.json**

Metrics / evaluation criteria

- 315 GT rows; tier counts NONE=304 / GRADUAL=6 / AEB=5 (2.7% GRADUAL+AEB overall)
- A zero vel_invalid-rate finding was checked against the full dataset population before being accepted as plausible rather than a bug.

Purpose. React to whichever sensor has new data right now, at that sensor's own real rate — not a fixed, synchronized tick that throws away timing information.

Logic used

- A lazy **heapq.merge** of each channel's own already-time-ordered sample_data chain into one globally chronological stream — $O(N \log k)$, never a full re-sort.

Inputs	Outputs
• A scene token • The set of active channels (DEC-3: LIDAR_TOP + 3 front radars + CAM_FRONT)	• An iterator of RawEvent records in true chronological order (metadata only — no files loaded yet)

Metrics / evaluation criteria

- Real per-channel inter-arrival-time statistics — camera measured at a precise 10.00 Hz (investigated and accepted as a real dataset property, not a bug).

STEP 8 — step06	Shared Short-Memory Tracker	APPROVED
--------------------	------------------------------------	----------

Purpose. One tracker, fed by every active sensor at once (early fusion) — never a separate per-sensor tracker merged together afterward.

Logic used

- Evict stale tracks, then predict every live track to the scan's timestamp (non-mutating peek), then Hungarian ($N \times M$) assignment of detections to tracks.
- Gating uses the **innovation covariance** $S = P_{\text{pred}} + R$ (Mahalanobis distance, chi-square threshold) — never a fixed Euclidean radius, and never P alone (must include the incoming detection's own noise).
- sensors_in_estimate is a **bounded recent window** (last 4 real updates), never a lifetime-growing set.

Inputs	Outputs
• list[Detection] for one scan • The scan's own timestamp	• list[TrackSnapshot] — one per currently-live track

Metrics / evaluation criteria

- Synthetic gating validation: $d^2(S=P+R)=0.626$ correctly accepted vs. $d^2(P \text{ alone})=29.195$ would have wrongly rejected the same real match.
- A genuine **track-identity-hijacking** failure mode was found and documented (not fixed) — two same-class objects close together can cause one track to silently swap which real object it represents.

STEP 9 — step05B	Track-Level Path Relevance	APPROVED
---------------------	-----------------------------------	----------

Purpose. Once a track has a real velocity, apply a stricter, motion-aware test: is it already in the corridor, or clearly about to enter it.

Logic used

- **IN_PATH** if already inside the strict corridor (confident, velocity-independent). **ENTERING** if extrapolating its own relative velocity forward (bounded by ENTERING_HORIZON_S) lands it inside the corridor while closing longitudinally.
- Falls back to the generous position-only candidate_gate (marked *provisional*) if the track's velocity isn't yet eligible to trust.

Inputs	Outputs
• TrackSnapshot • EgoState at the same instant	• RelevanceResult: verdict (IN_PATH / ENTERING / OUT) + a provisional flag

Metrics / evaluation criteria

- 6 synthetic tests, all passed on first implementation.

STEP 10 — step07B	Track-Level TTC Estimator & Critical-Object Selection	APPROVED
----------------------	--	----------

Purpose. Wire the pure TTC math to real tracks, and pick the single most urgent object to act on when several are path-relevant at once.

Logic used

- Eligibility reuses step05B's own `is_velocity_eligible` — one implementation, never duplicated.
- **critical_object()**: among path-relevant tracks, pick the smallest finite TTC. Never gated by object class (DEC-2, symmetric across all four sensor comparisons).

Inputs	Outputs
• TrackSnapshot • EgoState at the same instant (causally — must be the ego state AT that exact snapshot time, never a later/different one)	• TTCResult per track • The single (track_id, TTCResult) the system will act on

Metrics / evaluation criteria

- Causality test: proves feeding a later/faster/closer ego state gives a materially different (wrong) answer, confirming the causal design actually matters.

STEP 11 — step08	Debounce & Action Tiers	APPROVED
---------------------	------------------------------------	----------

Purpose. A single frame's TTC can be wrong (one bad match, one noisy return). Real AEB systems require the danger condition to hold for several consecutive real updates before acting.

Logic used

- **ActionTierDebouncer** requires DEBOUNCE_N consecutive real updates before a tier (GRADUAL / AEB) fires, and the same number of consecutive safe updates before it releases.
- A change in which track is the "critical" one resets the streak — never debounces across two different real objects.

Inputs	Outputs
• A timestamp + the current critical TTCResult (or None if nothing qualifies)	• ActionRecord: tier, driving_track_id, consecutive_count, a no_estimate flag • LatencyRecord (time from first danger reading to the tier actually firing)

Metrics / evaluation criteria

- 10 synthetic tests. Two real logic bugs found and fixed: consecutive_count silently staying 0 until a tier fired (fixed to show in-progress streak), and a latency-test assumption that only one tier fires per sequence (both AEB and GRADUAL

legitimately can fire together — fixed the test, not the code).

STEP 12 —
step09

Bounded Causal Extrapolation to Ground-Truth Instants

APPROVED

Purpose. Ground truth only exists at 2 Hz keyframes; the pipeline updates at native sensor rate. Answer, fairly: what did the system believe at exactly the GT instant, using only information available up to that instant.

Logic used

- Reconstructs a throwaway KF seeded from a track's last real snapshot and calls its own, already-tested **state_at()** to project forward — reusing the exact tracker math, never a second, parallel "pos + v*dt" implementation.
- Two hard bounds: never extrapolate backward before a track's first real update, and never extrapolate forward beyond EVICTION_GAP_S (the track would already be evicted by then).

Inputs

- A track's own snapshot history
- The GT keyframe timestamp
- A causal EgoStateEstimator

Outputs

- **SystemView**: per-track extrapolated state, the critical object (if any), the action decision at that instant, and a no_estimate_reason when nothing qualifies

Metrics / evaluation criteria

- Bounding-guard tests (confirms no backward and no unbounded extrapolation is possible).

STEP 13 —
step10

Event-Level Metrics & Scoring

APPROVED

Purpose. Score the system the way an AEB engineer would: at each real moment something was in the path, did the system react correctly — not full-track MAE/RMSE, not MOT metrics.

Logic used

- A 3×3 confusion matrix (GT tier × system tier: NONE/GRADUAL/AEB) drives every rate: correct_action_rate, false_brake_rate, missed_brake_rate, under/over-brake counts, phantom-brake rate (empty-corridor keyframes).
- Every rate reported with its raw numerator/denominator and a **Wilson score 95% confidence interval** — validated against published reference values before use.
- Every single-sensor result computed twice: unrestricted, and restricted to only the GT events that sensor could plausibly have observed at all (an observability variant) — so a sensor is never blamed for a genuine physical blind spot.
- A hard, code-level **eval-split guard**: refuses to score the held-out eval scenes without an explicit --final flag AND a matching frozen-config SHA-256 hash; every attempt, successful or refused, is permanently logged.

Inputs

- list[KeyframeGT] (the answer key)
- dict[sample_token → KeyframeSystem] (what the system believed at each GT instant)

Outputs

- **RunMetrics** (every rate + CI + confusion matrix + reason breakdown)
- **outputs/tables/four_run_table.*** (the 4-row comparison, step10's own generator)

Metrics / evaluation criteria

- This step is the metrics/evaluation layer — its own Wilson-interval implementation checked against two published reference values before being trusted.

Purpose. Turn a raw LiDAR point-cloud sweep into a short list of object candidates with a position and a noise estimate — nuScenes gives points, not detections.

Logic used

- Load sweep → transform sensor→ego → coarse ROI crop (compute-only, not the real decision boundary) → RANSAC ground-plane removal (fit on near-field low points, applied to the whole ROI) → DBSCAN clustering (eps=0.7m, min_samples=10, ported from the predecessor project's own tuned values) → one Detection per cluster, using the **nearest-surface-to-ego** reference point (DEC-1) — not the cluster centroid.
- A real sensor artifact was found and fixed: the LiDAR's own roof mount reflecting its beam back at point-blank range produced a 9,805-point phantom "object"; fixed with a minimum-range exclusion, verified across multiple real scenes.

Inputs	Outputs
• RawEvent (LIDAR_TOP) • The raw .bin point-cloud file	• list[Detection] in the global frame, each with a range-independent covariance R

Metrics / evaluation criteria

- 7 synthetic tests. First real LiDAR-only tune-split run: 94.9% correct_action_rate, 0% false_brake_rate; the one missed-brake finding traced to a genuine sensor blind spot (near-zero LiDAR points on that specific object), not a bug.

Purpose. Radar returns several points per real object per scan; cluster them into one detection per object, deliberately without using the sensor's own instantaneous-velocity reading.

Logic used

- DBSCAN per channel, per scan (eps=1.5m — wider than LiDAR's, since radar returns are sparser — min_samples=1, since radar often gives only one point per real object).
- Uses only the devkit's own default point filters — a deliberate choice NOT to add an extra stationary-clutter filter, so the downstream pipeline (corridor, tracker, debounce) is honestly tested against real "ghost" returns rather than having them hidden upstream.
- Doppler velocity is recorded in aux data only, never fed into the tracker or TTC math (a deliberately conservative baseline, revisitable later with explicit approval).

Inputs	Outputs
• RawEvent (one of the 3 active radar channels) • The raw .pcd point-cloud file	• list[Detection], Doppler/RCS/stationary-flag data carried in aux only

Metrics / evaluation criteria

- 7 synthetic tests. First real radar-only run found one false brake (1.3%), fully traced to a real, physically-present construction barrier — a genuine "ghost return" example, exactly the phenomenon this step is designed to measure honestly.

Purpose. A camera gives a 2D box, never a distance. Recovering distance must come from the camera alone — the predecessor project's own "camera" baseline secretly borrowed LiDAR depth, making its result fake; this step is built to make that mistake impossible.

Logic used

- A pretrained detector (Ultralytics YOLOv8n, CPU inference, results pre-cached — inference is slow) produces 2D boxes; truncated-bottom boxes (ground contact unknown) are dropped and counted.
- Depth via **ground-plane back-projection** (DEC-6): the box's bottom-centre pixel → a camera ray via the event's own intrinsics → rotated to the ego frame via the event's own extrinsics → intersected with an assumed flat ground plane.
- R grows with range² (a fixed pixel error at the box's bottom edge maps to a rapidly growing ground-distance error at long range/grazing angles) — empirically confirmed.
- A structural **isolation test** statically scans this module's own source and imports for any reference to the other sensors, making the predecessor project's exact mistake impossible to reintroduce.

Inputs

- RawEvent (CAM_FRONT)
- Cached 2D detector output (box, class, confidence)

Outputs

- list[Detection] with a range-dependent covariance R

Metrics / evaluation criteria

- 9 synthetic tests. Real-data depth error measured by range band: 1.78m median at 0–15m, growing to 17.29m at 50–200m — a direct empirical confirmation of the range² limitation.

Purpose. Run the identical pipeline four times — LiDAR-only, radar-only, camera-only, fused — tune shared settings on the tuning scenes only, freeze everything, then score the held-out scenes exactly once. Fusion's benefit is a claim to test, not an assumption.

Logic used

- "Fused" is not a separate pipeline: it is the identical code, with all three adapters feeding into the **same shared tracker** at once (early fusion).
- The tuning-value selection criterion (DEC-7: mean correct_action_rate across all four streams, missed_brake_rate as tie-breaker) was **pre-declared before any sweep ran**, so it could not be chosen afterward to favor fusion.
- A log-spaced grid sweep selected the shared process-noise value (SIGMA_A); three further thresholds were measured directly from real tune-split data — with three real bugs found and fixed in that measurement methodology itself before any value was trusted.
- The config is frozen (SHA-256 hash) and human-approved *before* the held-out eval split is ever touched; it is then scored exactly once, enforced by step10's own code-level guard.

Inputs

- The tune split (5 scenes) for tuning
- The held-out eval split (5 scenes) for the one final score
- The frozen, human-approved config

Outputs

- outputs/tuning_log.csv, configs/frozen_config.json, outputs/tables/four_run_table.*, reports/final_report.md, docs/knowledge_share.md

Metrics / evaluation criteria

- **The project's own headline evaluation criteria:** correct_action_rate, false_brake_rate, and missed_brake_rate (each with Wilson 95% CIs), compared across all four streams. Final result: LiDAR alone scored best overall (97.0% correct, 0.8% false); fused tied LiDAR for the best missed-brake rate (28.6%) but had the worst false-brake rate (11.1%) — a real, investigated trade-off, not a bug.

Overall Evaluation Criteria (the final scoring layer)

Every one of the four final runs (lidar / radar / camera / fused) is scored identically, using step10's own event-level metrics, at every ground-truth keyframe with at least one real in-path object:

- **correct_action_rate** — did the system's action tier (NONE/GRADUAL/AEB) exactly match the real danger tier at that instant.
- **false_brake_rate** — among keyframes with no real danger, how often did the system brake anyway.
- **missed_brake_rate** — among keyframes with real danger, how often did the system fail to react at all.
- **under/over-brake counts** — reacted, but at the wrong severity tier.
- **phantom_brake rate** — false reactions on keyframes with an entirely empty corridor.
- Every rate is reported with its raw numerator/denominator and a 95% Wilson score confidence interval, and flagged "indicative only" when the denominator is small (below 10).
- Every single-sensor rate is additionally computed restricted to only the GT events that sensor could plausibly have observed at all — so a sensor's own physical blind spot is never scored as a tracking failure.

Final result table (held-out eval split, scored once):

Stream	correct_action_rate	false_brake_rate	missed_brake_rate
LiDAR	97.0% (129/133)	0.8% (1/126)	28.6% (2/7)
Radar	94.0% (125/133)	3.2% (4/126)	42.9% (3/7)
Camera	95.5% (127/133)	0.8% (1/126)	71.4% (5/7)
Fused	87.2% (116/133)	11.1% (14/126)	28.6% (2/7)

Reading it honestly: LiDAR alone scored best on both correctness and false alarms. Fused tied LiDAR for the best missed-brake rate (a genuine benefit of combining sensors with different blind spots) but had by far the worst false-brake rate — traced (not just accepted) to roughly half its false alarms coming from one crowded scene, a known, already-documented tracker limitation (crowd conflation among closely-spaced same-class objects), not a new defect. Missed-brake rates are built on only 4–7 real events and are explicitly indicative only. Full investigation: reports/final_report.md.

Glossary

TTC

Time-to-Collision — seconds until the ego vehicle would reach the nearest object ahead, at current relative speed.

Ego vehicle

The vehicle the sensors are mounted on ("self").

GT

Ground Truth — the real, human-annotated answer key nuScenes provides.

KF

Kalman Filter — a statistical method for estimating a moving object's position/velocity from noisy measurements over time.

R (sensor noise)

How much a sensor's own measurement is expected to wobble around the true value; a Kalman filter needs this number to know how much to trust each reading.

Gating (Mahalanobis)

Deciding whether a new detection could plausibly belong to an existing track, accounting for both the track's own uncertainty and the sensor's own noise — not just raw distance.

DBSCAN

A clustering algorithm that groups nearby points into one object without needing to be told how many objects exist in advance.

RANSAC

A robust curve/plane-fitting method that tolerates a large fraction of noisy/outlier points — used here to find the road's ground plane in a LiDAR sweep.

Early fusion

Combining detections from every sensor into ONE shared tracker, rather than tracking each sensor separately and merging the results afterward (late fusion).

Debounce

Requiring a condition to hold for several consecutive real readings before acting on it, to avoid reacting to one noisy blip.

Wilson score interval

A statistically sound way to compute a confidence interval for a rate/percentage, more reliable than the naive method on small sample counts.

Correct / false / missed brake rate

The three headline scoring metrics: did the system correctly match the real danger level (correct), did it brake when there was nothing dangerous (false), did it fail to brake when there genuinely was danger (missed).

Tune split / Eval split

The tune split is used for all setting/parameter choices; the eval split is held out, untouched, and scored exactly once at the very end for the real, final result.

PLACEHOLDER / ASSUMED / MEASURED

This project's own honesty labels for every constant: not yet justified at all (PLACEHOLDER), a reasoned choice with a stated justification (ASSUMED), or independently measured against real data with a stated confidence (MEASURED).